

Impact of Deep Compression Techniques on Phase-Specific Accuracy in TinyTCN for Real-Time Gait Detection

Nontakorn Pulakkeaw

Computer Engineering

Mahidol University

Phra Nakhon Si Ayutthaya, Thailand

nonthakorn.dev@gmail.com

Abstract—Wearable-based gait phase detection is a critical enabler for rehabilitation monitoring, fall-risk assessment, and assistive robotics, yet deploying deep learning models on resource-constrained microcontrollers remains challenging. This study investigates the impact of deep compression techniques—post-training INT8 quantization and structured channel pruning—on the phase-specific accuracy of a Temporal Convolutional Network (TinyTCN) designed for real-time gait phase classification on the ESP32-C3 microcontroller. Using shank-mounted IMU data from the NONAN GaitPrint dataset, subject-wise cross-validation was used to evaluate generalization. We systematically apply INT8 post-training quantization and structured channel pruning to TinyTCN, quantify phase-specific accuracy degradation (LR, LS, PSw, Sw), and map the efficiency–accuracy trade-off via a Pareto front of macro F1-score versus on-device model size. On the held-out test set (2.58M windows), INT8 quantization reduced estimated payload from 42.8 KB to 12.7 KB with only a 0.07 percentage-point drop in macro F1 (91.50% to 91.43%), whereas structured 50% pruning combined with INT8 reduced macro F1 by 3.05 pp (to 88.45%) with the largest F1 loss in PSw (−3.43 pp). On ESP32-C3, INT8+Prune50 achieved 69.4 ms mean inference latency with an 11.6 KB TFLite model, a 120 KB tensor arena, and ~156 KB minimum free heap remaining.

Index Terms—TinyTCN, deep compression, quantization, pruning, gait phase detection, ESP32-C3, edge AI, wearable sensors

I. INTRODUCTION

Accurate detection of gait phases—such as loading response, mid-stance, pre-swing, and swing—is fundamental to applications in clinical rehabilitation, fall prevention, prosthetic control, and sports biomechanics. Recent advances in deep learning, particularly recurrent and convolutional architectures, have substantially improved gait phase classification accuracy using wearable inertial measurement unit (IMU) sensors. However, these models are typically designed and validated on desktop or server-class hardware, and their large parameter counts and floating-point computations make them impractical for direct deployment on low-power, low-memory microcontrollers used in real-world wearable devices.

Temporal Convolutional Networks (TCNs), which leverage dilated convolutions to capture long-range temporal dependencies with fewer parameters than recurrent architectures, offer a promising middle ground between accuracy and efficiency. A

compact variant—referred to here as TinyTCN—can, in principle, be further compressed via quantization and pruning to meet the strict memory and latency budgets of microcontroller-class hardware such as the ESP32-C3.

While prior work has explored model compression broadly for time-series classification, relatively little attention has been paid to how compression affects accuracy unevenly across different gait phases. Transition phases such as Loading Response (LR) and Pre-Swing (PSw) are inherently more ambiguous due to overlapping biomechanical signal characteristics, and we hypothesize that these phases are more vulnerable to the information loss introduced by aggressive quantization and pruning than steady-state phases such as Swing (Sw).

This study addresses this gap by:

- 1) Designing and training a TinyTCN architecture for gait phase classification using shank IMU data from the NONAN GaitPrint dataset;
- 2) Systematically applying INT8 post-training quantization, as well as structured channel pruning, and quantifying their impact on phase-specific accuracy and F1-score, rather than overall accuracy alone;
- 3) Deploying the compressed models on ESP32-C3 hardware and measuring on-device inference latency and SRAM headroom.

The remainder of this paper is organized as follows: Section II reviews related work on gait phase detection, TCN architectures, and edge AI compression. Section III describes the methodology, including dataset preparation, model architecture, and compression protocols. Section IV presents experimental results. Section V discusses the implications of phase-specific degradation and hardware feasibility, and Section VI concludes the paper.

II. LITERATURE REVIEW

A. Gait Phase Detection Using Wearable IMU Sensors

Gait phase detection using body-worn IMU sensors has been extensively studied, with shank- and foot-mounted sensors commonly used due to their high sensitivity to phase transitions. Classical machine learning approaches (e.g., threshold-

based, hidden Markov models) have largely been superseded by deep learning methods, including 1D Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, hybrid LSTM+GRU architectures, and—more recently—Transformer encoders, which achieve higher accuracy by learning temporal dependencies directly from raw or lightly processed sensor signals. Using the same NONAN GaitPrint dataset and shank-mounted, six-axis IMU configuration adopted in this study, Choi and Choi [1] directly compared a 1D CNN, a hybrid LSTM+GRU model, and a Transformer for four-class gait phase classification, reporting comparable performance across the three architectures (macro F1 approximately 91–93%) with the Transformer attaining the highest score. This paper adopts the same three architectures, plus a standard TCN, as FP32 baselines under an identical protocol (Section III), but targets a distinct research question left open by prior work: how INT8 quantization and structured pruning affect *phase-specific* accuracy, and how the resulting models perform when deployed on microcontroller-class hardware.

B. Temporal Convolutional Networks (TCNs) for Time-Series Classification

TCNs use causal, dilated convolutions to achieve large receptive fields with a fraction of the parameters required by recurrent architectures, while supporting parallelized computation [2]. This property makes TCNs particularly attractive for edge deployment, where both parameter count and inference latency are critical constraints. Compact TCN variants (“TinyTCN”) extend this efficiency further by reducing channel width and depth while preserving sufficient receptive field coverage for a full gait cycle.

C. Model Compression Techniques: Quantization and Pruning

Post-training quantization reduces numerical precision (e.g., from 32-bit floating point to 8-bit integer representations), substantially reducing model size and enabling faster, more energy-efficient inference on integer-optimized microcontroller hardware [3]. Structured pruning removes entire channels or filters, yielding actual speedups on commodity hardware (unlike unstructured pruning, which often requires specialized sparse-matrix support) [4]. Both techniques have been widely studied for image and audio models, but their differential effect across distinct temporal phases of a periodic signal—such as gait phases—remains comparatively underexplored.

D. Edge AI Deployment on Microcontrollers (ESP32-C3)

TensorFlow Lite Micro (TFLM) enables compact neural networks to run directly on microcontroller hardware with kilobyte-scale memory budgets [5]. The ESP32-C3 is a single-core RISC-V microcontroller (160 MHz) with approximately 400 KB of SRAM [6], making it a low-cost target for TinyML deployment. Although it lacks the vector acceleration available on higher-end Espressif parts, compact INT8 models can still run on-device when combined with structured pruning, motivating its use as the deployment platform in this study.

E. Research Gap

While compression-accuracy trade-offs are well documented at the aggregate level, the literature provides limited insight into which specific gait phases are most degraded by compression. This is a meaningful gap because transition phases (LR, PSw) are often clinically and functionally the most informative (e.g., for fall-risk detection), yet may also be the most fragile under information loss. This paper directly addresses this gap.

III. METHODOLOGY

A. Dataset and Preprocessing

- **Dataset:** NONAN GaitPrint [7], using shank-mounted IMU channels (accelerometer and gyroscope axes).
- **Sampling rate:** Original 200 Hz signal downsampled to 100 Hz for all reported experiments, balancing temporal resolution against on-device resource constraints.
- **Labeling:** Gait cycle phases labeled as Loading Response (LR), Loading Stance (LS), Pre-Swing (PSw), and Swing (Sw).
- **Data split:** Subject-wise split (24 train / 5 validation / 6 test subjects), with no subject leakage across partitions.
- **Normalization:** Per-channel z-score normalization computed on the training set only.
- **Windowing:** Causal 0.5 s windows with train stride 5 and validation/test stride 1.
- **Training:** Adam optimizer ($\text{lr} = 10^{-3}$), batch size 512, up to 50 epochs; best checkpoint selected by validation macro F1 (TinyTCN best epoch: 48).
- **Evaluation:** 2,583,680 test windows; metrics reported as accuracy, macro F1, and phase-specific F1/accuracy.

Phase support is highly imbalanced (LS and Sw together account for $(1,274,499 + 1,016,857)/2,583,680 \approx 88.7\%$ of test windows, vs. only 11.3% for the transition phases LR and PSw combined). To prevent the majority phases from dominating optimization, all models—including the four FP32 baselines and every compression fine-tuning stage—are trained with a class-balanced weighted cross-entropy loss:

$$\mathcal{L} = - \sum_{c=1}^4 \alpha_c y_c \log \hat{y}_c, \quad \alpha_c = \frac{N}{4 \cdot N_c}, \quad (1)$$

where N is the number of training windows, N_c is the number of windows labeled with phase c , y_c is the one-hot ground-truth indicator, and $\hat{y}_c$ is the softmax output probability for phase c . This directly motivates reporting *macro*, rather than micro-averaged, metrics throughout this paper.

B. TinyTCN Architecture

The proposed TinyTCN consists of a small stack of dilated causal convolutional blocks (dilation rates 1, 2, and 4) with residual connections, batch normalization, and a lightweight classification head (global pooling followed by a dense softmax layer over the four phase classes). Each block computes a dilated causal convolution [2] (kernel size k , dilation d , ReLU) so that each output depends only on present and

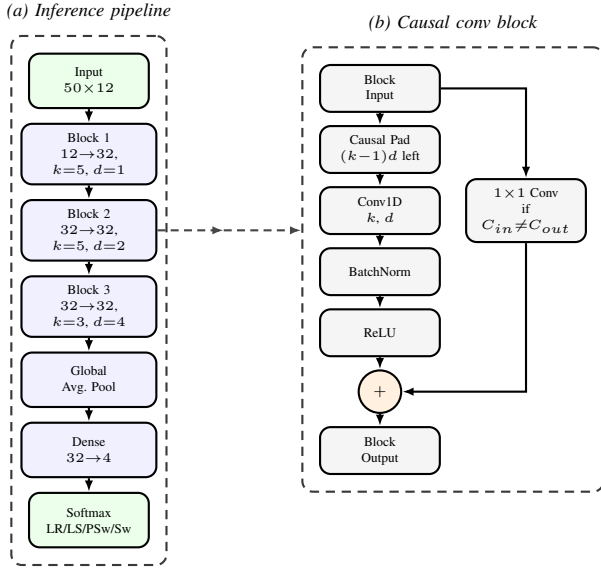


Fig. 1. TinyTCN architecture: (a) end-to-end pipeline from a 50×12 causal window to a 4-class softmax, and (b) internal structure of each causal convolutional block (left-padding, dilated Conv1D, batch norm, ReLU, and residual connection).

past inputs. Stacking three such blocks (kernel sizes 5, 5, 3; dilations 1, 2, 4) yields a receptive field of 21 samples (0.21 s at 100 Hz), comfortably within the 0.5 s (50-sample) input window while keeping the parameter count low. The model consumes causal 0.5-second windows (50 samples at 100 Hz). Channel width is deliberately kept narrow (32 hidden channels) to keep the model near 11K parameters while retaining temporal modeling capacity. Fig. 1 illustrates the overall data flow and the internal structure of each causal convolutional block.

C. Baseline Architectures for Context

The primary comparison in this work is across compression configurations of TinyTCN (FP32 vs. INT8 vs. pruned). For context, four FP32 baselines (CNN1D, TCN, LSTM+GRU, Transformer) were retrained under an identical protocol (Table II).

D. Deep Compression Protocol

Post-training quantization (INT8). We apply symmetric 8-bit quantization separately to each convolutional and fully connected weight layer. For every layer, a single scale is set from that layer’s largest absolute weight; each weight is rounded to the nearest integer in $[-127, 127]$, then mapped back to floating point for inference. This quantize–dequantize (QDQ) proxy is used for PyTorch-side test-set evaluation (Section IV). For ESP32 deployment, models are exported as full-integer TensorFlow Lite graphs using TFLite’s built-in post-training quantization.

Structured channel pruning (Prune50). TinyTCN uses 32 hidden channels in its convolutional stem. Each channel is ranked by the total absolute weight magnitude summed

TABLE I
DEEP COMPRESSION TECHNIQUES APPLIED TO TINYTCN

Technique	Variant	Description
Quantization	INT8 (post-training)	PyTorch QDQ proxy for test-set evaluation; full-integer TFLite PTQ for ESP32 export
Pruning	Structured, 50% channels	Removal of least-salient convolutional channels for real hardware speedup
Combined	INT8 Pruned	+ Joint compression for maximum efficiency

over all three stem convolutional blocks; the 16 highest-ranked channels are retained and the rest are removed from every stem layer and from the classification head. The pruned model is fine-tuned for 5 epochs (Adam, learning rate 10^{-4}) with the same class-balanced loss (Eq. 1) before re-evaluation.

Table I summarizes the compression techniques applied to TinyTCN.

E. Hardware Deployment (ESP32-C3)

After PyTorch compression evaluation, selected models are converted to TensorFlow Lite (.tflite) format and deployed via TensorFlow Lite Micro on an ESP32-C3 development board (160 MHz, Arduino IDE, Chirale_TensorFlowLite). On-device inference latency is measured per 0.5-second input window over 1000 timed trials (20 warmup invocations), alongside minimum free heap during benchmarking and a fixed 120 KB tensor arena.

F. Evaluation Metrics

Macro F1 is the unweighted mean of the four per-phase F1 scores. Unlike accuracy or a support-weighted average, this gives Loading Response (LR) and Pre-Swing (PSw) equal weight to the majority phases despite their $\sim 6\%$ and $\sim 5\%$ test-set support, respectively.

- **Phase-specific F1-score and accuracy** for LR, LS, PSw, and Sw individually.
- **Pareto analysis:** macro F1-score vs. measured or estimated model size (KB) per compression config.
- **Model complexity:** parameter count and payload size.
- **On-device performance:** inference latency (ms) and minimum free heap (KB) on ESP32-C3.
- **Efficiency-accuracy trade-off** visualized via a Pareto front across compression configurations.

IV. RESULTS

A. Baseline Comparison and Model Complexity

Table II compares TinyTCN against four FP32 baselines trained under an identical protocol (12-channel bilateral shank IMU, 100 Hz, subject-wise split). TinyTCN achieves the highest test macro F1 (91.50%) while remaining the smallest full-precision model among the compared architectures except CNN1D. The per-phase F1 rows in Table II show this

TABLE II
BASELINE COMPARISON ON THE HELD-OUT TEST SPLIT (FP32, FAIR TRAINING PROTOCOL)

	TinyTCN	Trans.	LSTM+GRU	TCN	CNN1D
Params	10,948	25,956	12,356	11,560	10,727
Size (KB)	42.8	101.4	48.3	45.2	41.9
Acc. (%)	96.30	96.38	96.25	96.04	95.58
Macro F1 (%)	91.50	91.73	91.38	90.96	90.23
<i>Phase-specific F1 (%)</i>					
LR	87.26	87.28	87.85	86.40	85.77
LS	97.88	98.07	97.91	97.71	97.54
PSw	82.99	83.84	82.00	82.01	80.29
Sw	97.87	97.72	97.78	97.73	97.34

TABLE III
TINYTCN COMPRESSION RESULTS AND PHASE-SPECIFIC F1 ON THE TEST SPLIT

	FP32	INT8	Prune50	INT8+Prune50
Params	10,948	10,948	3,428	3,428
Est. size (KB)	42.8	12.7	13.4	5.4
TFLite (KB)	49.2	18.9	20.3	11.6
Acc. (%)	96.30	96.27	94.74	94.56
Macro F1 (%)	91.50	91.43	88.73	88.45
<i>Phase-specific F1 (%)</i>				
LR	87.26	87.04	81.32	80.52
LS	97.88	97.84	96.88	96.76
PSw	82.99	82.95	79.67	79.56
Sw	97.87	97.87	97.03	96.95

advantage is not uniform across phases: Transformer attains the highest F1 on LS and PSw, and LSTM+GRU on LR, while TinyTCN leads only on Sw—indicating that TinyTCN’s overall macro F1 advantage stems from a more balanced profile rather than dominance on any single phase.

B. Compression Trade-off and Phase-Specific F1

Table III summarizes PyTorch-side compression evaluation on the same test split, reporting model complexity, exported TFLite size, overall accuracy, and macro/phase-specific F1 together. INT8 weight quantization preserved accuracy and macro F1 almost exactly, while structured pruning reduced parameter count to 3,428 (−68.7%) at a larger macro F1 cost. Exported TFLite sizes were measured for ESP32 deployment. PSw—a brief transition phase—showed the largest F1 degradation under pruning (−3.32 pp for Prune50 and −3.43 pp for INT8+Prune50 relative to FP32), whereas INT8 quantization alone changed each phase F1 by at most 0.04 pp.

C. Phase-Specific Accuracy Under Compression

Table IV reports phase-specific accuracy (overall test accuracy per config already appears in Table III and is omitted here to avoid duplication). Phase accuracy can diverge from F1 when precision drops but recall remains high, notably for LR under pruning. This motivates reporting both metrics in gait-phase studies.

TABLE IV
PHASE-SPECIFIC TEST ACCURACY (%) UNDER COMPRESSION

Config	LR	LS	PSw	Sw
FP32	95.03	96.38	91.67	97.03
INT8	95.20	96.28	91.18	97.11
Prune50	97.39	94.36	94.11	94.90
INT8+Prune50	97.62	94.10	94.41	94.71

TABLE V
ON-DEVICE INFERENCE PERFORMANCE ON ESP32-C3 (160 MHZ, TENSOR ARENA 120 KB, 1000 TRIALS)

Config	Latency (ms)	TFLite (KB)	Min Heap (KB)
FP32	850.6	49.2	156
INT8	256.8	18.9	156
Prune50	260.0	20.3	156
INT8+Prune50	69.4	11.6	156

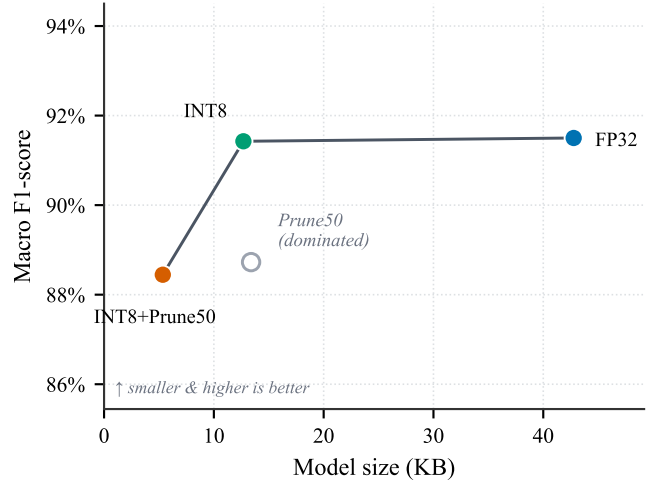


Fig. 2. Pareto front of macro F1-score vs. deployment size (KB). Filled markers connected by the solid line are Pareto-optimal; Prune50 (hollow marker) is dominated by INT8, which achieves both a smaller footprint and higher macro F1.

D. On-Device Performance on ESP32-C3

Table V reports latency from 1000 timed *Invoke()* calls on a fixed synthetic replay window (50×12, normalized with training-set statistics). Pruning alone did not reduce latency versus FP32 float on this board (∼260 ms), whereas INT8+Prune50 reduced latency to 69.4 ms—the only configuration practical for periodic inference at 50–100 ms intervals.

E. Pareto Front and Phase Degradation

V. DISCUSSION

A. Vulnerability of Transition Phases

Contrary to the initial hypothesis that LR would degrade first, PSw F1 showed the largest drop under structured pruning (−3.32 pp for Prune50 and −3.43 pp for INT8+Prune50),

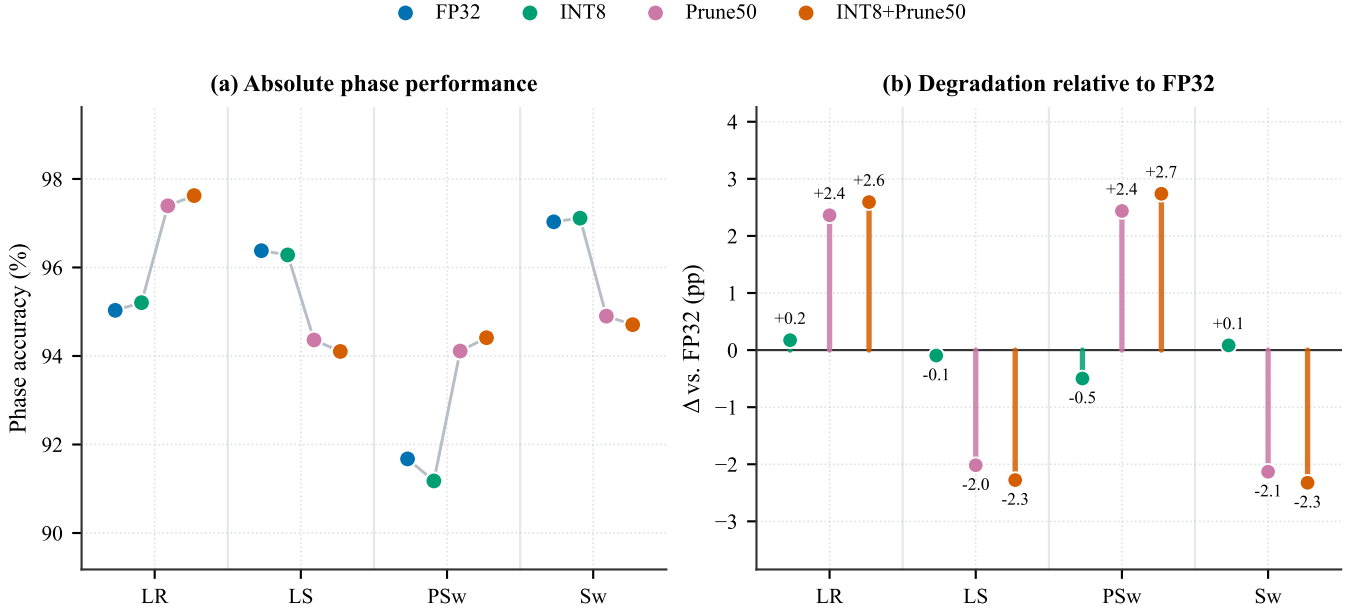


Fig. 3. Phase-specific test accuracy under compression. (a) Absolute per-phase accuracy for all four configurations, connected in order of increasing compression (FP32 $\rightarrow$ INT8 $\rightarrow$ Prune50 $\rightarrow$ INT8+Prune50). (b) Accuracy change relative to the FP32 baseline, highlighting the larger degradation in the transition phases LR and PSw under structured pruning.

while INT8 quantization alone changed phase F1 by at most 0.04 pp. PSw is a short transition window whose decision boundary likely depends on finer temporal detail removed when hidden channels are halved. Notably, LR accuracy increased under pruning (+2.36 pp, Table IV) even as LR F1 fell (Table III), indicating reduced precision despite higher recall—a pattern invisible when reporting accuracy alone.

B. Role of Dilated Convolutions Under Compression

INT8 quantization preserved macro F1 within 0.07 pp of FP32, suggesting that TinyTCN’s dilated receptive field and narrow hidden width (32 channels) remain sufficient under 8-bit weight precision. Pruning, by contrast, removes representational capacity and disproportionately affects the already difficult PSw class (baseline F1 82.99%, the lowest among the four phases).

C. Efficiency-Accuracy Trade-off

The Pareto analysis (Fig. 2) shows INT8 as the dominant accuracy-preserving compression point (12.7 KB estimated, 91.43% macro F1). INT8+Prune50 occupies the smallest-size corner (5.4 KB estimated, 11.6 KB TFLite) at 88.45% macro F1. For deployment, this trade-off must be paired with on-device latency: on ESP32-C3, only INT8+Prune50 achieved sub-100 ms inference (69.4 ms mean), whereas full-width INT8 required 256.8 ms despite its superior accuracy.

D. Feasibility for Real-World Wearable Deployment

At 69.4 ms per window, INT8+Prune50 is suitable for coaching and logging use cases when inference is scheduled every 50–100 ms (stride 5–10 at 100 Hz). It does not satisfy

a strict 100 Hz streaming budget (10 ms). Host-side TFLite smoke tests confirmed matching argmax predictions for all exported configs on a fixed replay window, supporting the validity of the on-device benchmark pipeline.

E. Limitations

- Results are derived from a single public dataset (NONAN GaitPrint); generalization to other populations, walking speeds, or pathological gait patterns requires further validation.
- On-device benchmarks used a synthetic replay window, not live IMU streaming; end-to-end latency with BLE acquisition remains unmeasured.
- PyTorch INT8 evaluation uses a quantize-dequantize weight proxy; deployed INT8 TFLite models use full-integer PTQ and may differ slightly in logits while preserving argmax in our smoke tests.
- Power consumption (mW) was not measured on ESP32-C3.
- Latency used TFLite Micro reference kernels (Chirale_TensorFlowLite); optimized INT8 kernels could reduce inference time further.

VI. CONCLUSION

This study systematically evaluated how INT8 quantization and structured 50% channel pruning affect phase-specific accuracy in TinyTCN for gait phase detection. On 2.58M test windows, INT8 preserved 91.43% macro F1 (vs. 91.50% FP32) at roughly one-third the estimated payload size, whereas INT8+Prune50 reduced size to 5.4 KB (11.6 KB TFLite) at 88.45% macro F1 with the largest F1 loss concentrated in

PSw. Disaggregating metrics by phase revealed that pruning can inflate phase accuracy while lowering F1—an important distinction for transition-phase monitoring. On ESP32-C3, INT8+Prune50 ran in 69.4 ms per 0.5 s window, demonstrating feasibility for low-cost wearable monitoring at moderate update rates. Future work should validate live IMU streaming, measure power, and extend the analysis to pathological gait populations.

ACKNOWLEDGMENT

The authors thank the creators of the NONAN GaitPrint dataset for making their IMU gait recordings publicly available, which made this study possible.

REFERENCES

- [1] W. Choi and M.-T. Choi, “Deep learning-based gait phase detection using shank-mounted IMU data: Classification approach,” *PLOS ONE*, vol. 21, no. 4, p. e0344002, 2026, doi: 10.1371/journal.pone.0344002.
- [2] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.
- [3] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [4] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.
- [5] R. David et al., “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.
- [6] Espressif Systems, *ESP32-C3 Technical Reference Manual*, version 1.3, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf
- [7] T. M. Wiles et al., “NONAN GaitPrint: An IMU gait database of healthy young adults,” *Scientific Data*, vol. 10, no. 867, 2023, doi: 10.1038/s41597-023-02704-z.